Laurea Triennale in informatica-Università di Salerno Corso di Ingegneria del Software- Prof.ssa F. Ferrucci, Prof. F. Palomba

Design Patterns Document

C-09 SINTONIA

Riferimento	2025_C09_DPD
Versione	1.0
Data	21/11/2025
Destinatario	Prof.ssa Filomena Ferrucci, Prof.re Fabio Palomba
Presentato da	C09 Team Sintonia
Approvato da	//

SINTONIA
La mente trova pace, un passo alla volta

Revision History

Data	Versione	Descrizione	Autori
21/11/2025	1.0	Configurazione Documento	Alessio Del Sorbo
21/11/2025	1.0	Aggiunto Scopo del documento	Mattia Fanzini
21/11/2025	1.0	Aggiunto Introduzione Design Patterns	Francesco Di Giovanni
21/11/2025	1.0	Aggiunto Riferimenti	Gavino De Stefano
21/11/2025	1.0	Aggiunto Panoramica del Documento	Gianni Policola
21/11/2025	1.0	Aggiunto Design Pattern Facade	Gavino De Stefano
21/11/2025	1.0	Aggiunto Design Pattern Adapter	Gianni Policola
21/11/2025	1.0	Aggiunto Design Pattern Proxy	Francesco Coppola

SINTONIA
La mente trova pace, un passo alla volta

Team Members

Ruolo	Nome e Cognome	Acronimo	Contatto
PM	Gianfranco Barba	GB	g.barba14@studenti.unisa.it
PM	Francesco Corcione	FC	f.corcione5@studenti.unisa.it
TM	Gianluca Ambrosio	GA	g.ambrosio39@studenti.unisa.it
TM	Elena Carlomagno	EC	e.carlomagno1@studenti.unisa.it
TM	Rosaria Cervino	RC	r.cervino3@studenti.unisa.it
TM	Francesco Coppola	FCP	f.coppola99@studenti.unisa.it
TM	Antonio Walter De Fusco	ADF	a.defusco1@studenti.unisa.it
TM	Gavino De Stefano	GDS	g.destefano41@studenti.unisa.it
TM	Alessio Del Sorbo	ADS	a.delsorbo32@studenti.unisa.it
TM	Francesco Di Giovanni	FDG	f.digiovanni10@studenti.unisa.it
TM	Mattia Fanzini	MF	m.fanzini@studenti.unisa.it
TM	Angelo Fusco	AF	a.fusco58@studenti.unisa.it
TM	Gianni Policola	GP	g.policola@studenti.unisa.it

SINTONIA
La mente trova pace, un passo alla volta

Sommario

Sommario **4**

1 Introduzione **5**

1.1 Scopo del documento 5

1.2 Introduzione Design Patterns 6

1.3 Riferimenti 7

1.4 Panoramica del documento 8

2 Design Patterns **9**

2.1 Facade 9

2.2 Adapter 10

Per una visualizzazione ottimale, consultare il seguente link. 11

2.3 Proxy 12

Glossario **14**

SINTONIA
La mente trova pace, un passo alla volta

1 Introduzione

1.1 Scopo del documento

Il presente documento, denominato **Design Pattern Document (DPD)**, costituisce la specifica dettagliata delle scelte progettuali a livello di oggetto per il sistema **SINTONIA**. Esso rappresenta il passaggio logico successivo alla definizione dell'architettura di sistema descritta nel *System Design Document (SDD)* e si pone come guida tecnica per la fase di implementazione del software.

Mentre l'SDD ha definito l'architettura ad alto livello (Three-Tier), la decomposizione in sottosistemi e lo schema logico dei dati , lo scopo di questo documento è scendere nel dettaglio delle dinamiche interne ai componenti software, focalizzandosi su **come** i requisiti funzionali vengono soddisfatti attraverso l'adozione di soluzioni progettuali standardizzate.

Nello specifico, gli obiettivi di questo documento sono:

- **Documentare i Design Pattern:** Identificare e giustificare l'adozione di specifici pattern architetturali e comportamentali utilizzati per risolvere problemi ricorrenti di progettazione, garantendo modularità e manutenibilità, in linea con le scelte tecnologiche effettuate (NestJS per il Backend e React per il Frontend).

- **Definire le Interfacce delle Classi:** Dettagliare le *signature* dei metodi e le responsabilità delle classi appartenenti ai layer definiti nell'SDD (Controller, Service, DAO) ,arricchendo il Diagramma delle Classi definito nell'SDD con i dettagli implementativi necessari alla codifica.

- **Guidare l'Implementazione:** Fornire al team di sviluppo un riferimento univoco per la scrittura del codice, assicurando che le componenti software rispettino i principi di *Design Goals* stabiliti, quali la robustezza, la sicurezza dei dati sanitari e la manutenibilità del sistema.

- **Mappare le Interazioni:** Chiarire come gli oggetti interagiscono per realizzare gli scenari complessi descritti nel RAD, come il *Triage Dinamico* e la *Gestione degli Alert Clinici*, integrando le logiche dei *Sequence Diagrams* con le strutture dati definite.

Questo documento è destinato principalmente agli sviluppatori del team C09 e ai responsabili della manutenzione futura, fungendo da ponte tra l'analisi dei requisiti e il codice sorgente finale.

1.2 Introduzione Design Patterns

Nel sistema che si intende realizzare, è stata presa la decisione di adottare design pattern per affrontare in modo efficace le sfide architetturali e organizzative del sistema. L'uso di design pattern favorisce la creazione di un sistema modulare, flessibile e facilmente manutenibile, riducendo le dipendenze e migliorando la gestione delle interazioni tra le diverse componenti del sistema.

I pattern selezionati sono: **Facade, Adapter** e **Proxy**.

Sono descritti in questo modo:

- **Facade:** Il design pattern Facade viene utilizzato per semplificare l'interazione con sistemi complessi. In un contesto software, i sistemi tendono ad essere costituiti da molteplici componenti, ognuno dei quali può avere un'interfaccia e una logica complessa. L'uso di Facade ha lo scopo di nascondere questa complessità agli utenti finali o ad altri componenti del sistema, offrendo un'interfaccia semplice e coerente per l'accesso ai servizi principali.

  Nel nostro sistema, il Facade viene utilizzato per interfacciare il sottosistema di gestione clinica: fornisce un unico punto di accesso semplice e coerente, ne nasconde la complessità interna e mette a disposizione del resto della piattaforma operazioni chiare e uniformi.

- **Adapter:** Il pattern Adapter è utilizzato per risolvere problemi di incompatibilità tra interfacce. Quando due componenti o sistemi devono interagire ma hanno interfacce incompatibili, Adapter permette di "adattare" una delle interfacce in modo che possa essere utilizzata dal sistema di destinazione. In pratica, il pattern funge da ponte tra due sistemi, permettendo loro di lavorare insieme senza modificare il codice esistente.

  Nel nostro sistema, l'Adapter viene utilizzato per integrare moduli esterni, come il modulo di Intelligenza Artificiale, che potrebbero avere interfacce e formati di dati incompatibili con l'architettura del sistema. L'Adapter facilita la comunicazione tra il sistema principale e il modulo di IA, traducendo i dati e i metodi in un formato che il sistema può utilizzare senza problemi.

- **Proxy:** Il pattern Proxy è un design pattern strutturale che fornisce un surrogato o un segnaposto per un altro oggetto al fine di controllarne l'accesso. L'obiettivo è interporre un livello di controllo tra il client e l'oggetto reale, permettendo di eseguire verifiche (come controlli di sicurezza o validazione dei permessi) prima che la richiesta venga

effettivamente inoltrata. Questo è essenziale quando l'accesso diretto agli oggetti potrebbe compromettere la sicurezza o l'integrità dei dati.

Nel nostro sistema, il pattern Proxy viene utilizzato per proteggere l'accesso alle tabelle contenenti dati sensibili, come ad esempio **Pagina_Diario**, **Paziente, Psicologo, Amministratore** e **Questionario**. Poiché il database distingue tra l'account di accesso e i dati personali, il Proxy agisce come un guardiano: intercetta le richieste di lettura o scrittura (ad esempio, la visualizzazione di un diario personale) e verifica che l'utente autenticato abbia i diritti legittimi per accedere a quella specifica risorsa (es. il paziente proprietario del diario o lo psicologo a lui assegnato), bloccando tentativi di accesso non autorizzati prima che raggiungano il database.

1.3 Riferimenti

- *Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd Edition* - Bernd Bruegge & Allen H. Dutoit
- *Requirements Analysis Document* - C09 SINTONIA
- *System Design Document* - C09 SINTONIA

1.4 Panoramica del documento

Il presente documento è strutturato in due capitoli principali, che guidano il lettore dalla definizione degli obiettivi architetturali fino alla specifica tecnica delle soluzioni adottate.

- **Introduzione**: Definisce il contesto e gli obiettivi del documento, chiarendo lo scopo della progettazione di dettaglio e fornendo i riferimenti bibliografici necessari alla comprensione degli schemi proposti.

- **Design Patterns**: Costituisce il nucleo del documento e descrive le soluzioni architetturali applicate per risolvere le criticità di integrazione e flessibilità del sistema SINTONIA. Ogni pattern è analizzato attraverso il problema specifico, la soluzione adottata e il diagramma delle classi di riferimento. In particolare:

    - **Facade**: Descrive l'adozione di questo pattern per semplificare l'interazione tra l'interfaccia utente e la logica di business complessa del sottosistema di Gestione Clinica, fornendo un punto di accesso unificato per le operazioni dello psicologo.

    - **Adapter**: Illustra la strategia di integrazione del modulo di Intelligenza Artificiale (Python) all'interno dell'architettura backend (NestJS). Il pattern agisce da intermediario, separando la logica di business dalle complessità tecniche legate all'esecuzione del modulo di IA.

    - **Proxy:** Delinea il meccanismo di sicurezza strutturale implementato per proteggere l'accesso alle entità contenenti dati sensibili, quali **Paziente**, **Psicologo, Pagina_Diario, Amministratore** e **Questionario**. Il pattern interpone un livello di controllo che intercetta le richieste di lettura e scrittura, verificando la legittimità delle autorizzazioni dell'utente prima di consentire l'accesso effettivo ai dati riservati.

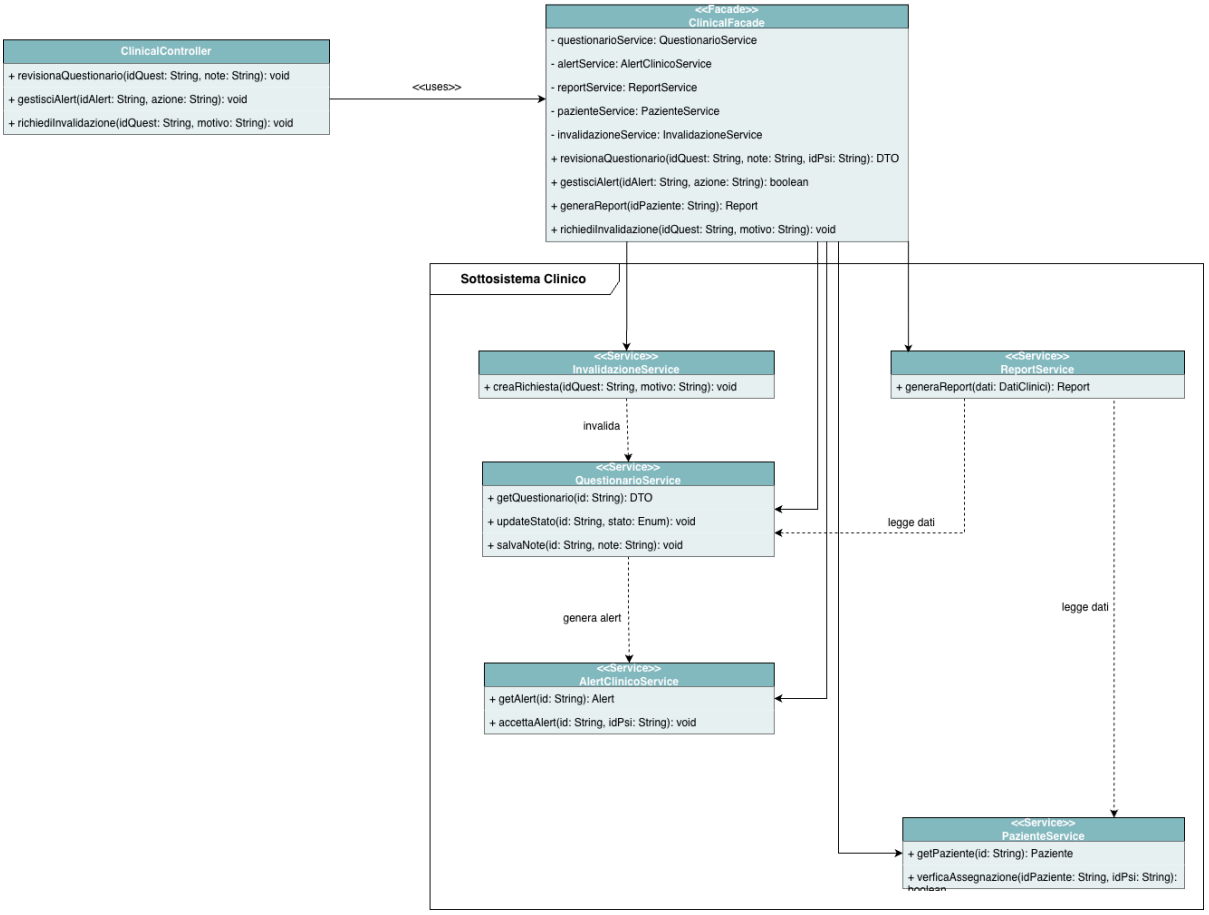
2 Design Patterns

2.1 Facade

Il design pattern Facade è un tipo di design pattern strutturale. Lo scopo principale di questo design pattern è fornire un'interfaccia unificata e di più alto livello per un insieme di funzioni di un determinato sottosistema: semplifica l'utilizzo delle interfacce, incapsulando la complessità del sottosistema.

L'utilizzo del design pattern permette il disaccoppiamento dei client dai dettagli implementativi del sottosistema, riducendo la dipendenza tra le classi e favorendo flessibilità e manutenibilità.

Nel caso di **SINTONIA**, verrà utilizzato per interfacciare il **Sottosistema di Gestione Clinica** con i Controller, fornendo un unico punto di accesso per le operazioni complesse.



Per una visualizzazione ottimale, consultare il seguente link.

2.2 Adapter

Nel contesto del sottosistema Gestione Clinica, il requisito funzionale **RF_PSI_5 (Generazione Report)** prevede l'elaborazione avanzata dei dati del paziente (es. annotazioni del diario, storico stati d'animo) per fornire allo psicologo un riepilogo clinico supportato da Intelligenza Artificiale.

Per sfruttare le più avanzate librerie di Natural Language Processing (NLP) e Data Science, il modulo di IA è stato sviluppato in ambiente **Python**. Tuttavia, il core del sistema SINTONIA è realizzato in **NestJS (TypeScript)**. La coesistenza di questi due stack tecnologici introduce un disallineamento architettonico su due livelli:

- **Incompatibilità di Comunicazione**: Il backend NestJS opera all'interno del proprio runtime (Node.js), mentre il modulo AI risiede in un processo esterno (o servizio separato) che richiede un protocollo di comunicazione dedicato (es. HTTP o esecuzione di processo), diverso dalla semplice chiamata di metodo locale.
- **Disallineamento dei Dati**: La logica di business gestisce oggetti di dominio tipizzati (DTO TypeScript), mentre il motore Python richiede input strutturati specifici (es. dizionari o JSON) e restituisce output grezzi che necessitano di parsing e normalizzazione.

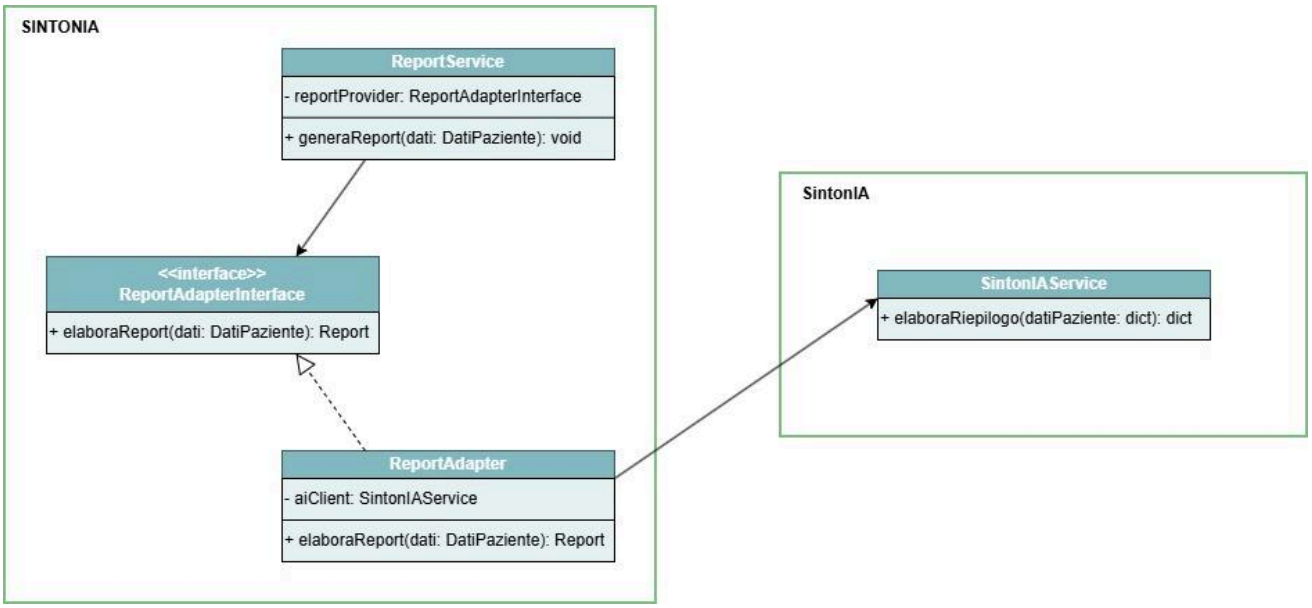
Un accesso diretto al componente Python all'interno della Business Logic violerebbe il principio di separazione delle responsabilità, creando un forte **accoppiamento tecnologico**. Ciò renderebbe il sistema rigido e difficile da manutenere in caso di future sostituzioni del motore di analisi.

Per risolvere tali criticità, è stato adottato il **Design Pattern Strutturale Adapter**. Nel sistema SINTONIA, l'Adapter agisce come un componente intermedio ("wrapper") che incapsula la complessità della comunicazione con il mondo Python. Esso espone al Service interno un'interfaccia TypeScript standard e stabile, facendosi carico internamente della trasformazione dei dati (mapping) e della gestione delle invocazioni verso il componente esterno.

Di seguito viene illustrata la modellazione UML della soluzione adottata:

- **Client (ReportService)**: Rappresenta la logica di business di SINTONIA. Grazie al principio di Dependency Inversion, esso non dipende dall'implementazione concreta dell'IA, ma interagisce esclusivamente con l'interfaccia astratta ReportAdapterInterface.

- **Target (ReportAdapterInterface)**: Definisce il contratto standard atteso dal sistema, richiedendo in input un oggetto di dominio tipizzato (DatiPaziente) e restituendo un oggetto Report.
- **Adapter (ReportAdapter)**: È la classe che implementa l'interfaccia e realizza l'adattamento vero e proprio. Al suo interno incapsula un'istanza del servizio esterno (SintonIAService). Il suo compito è convertire i DatiPaziente nel formato dizionario richiesto dal motore Python, inoltrare la richiesta e infine ricostruire l'oggetto Report a partire dalla risposta grezza.
- **Adapter (SintonIAService)**: Rappresenta il client verso il motore di Intelligenza Artificiale esterno, che espone l'interfaccia originale incompatibile con il resto del sistema.



Per una visualizzazione ottimale, consultare il seguente link.

2.3 Proxy

Nel contesto della persistenza dei dati di SINTONIA, il sistema gestisce informazioni eterogenee con livelli di riservatezza critici. Oltre alle credenziali di accesso, il database ospita tabelle altamente sensibili quali **Paziente** (dati anagrafici e clinici), **Psicologo** (dati professionali), **Pagina_Diario** (riflessioni personali del paziente da mantenere al sicuro per privacy), **Questionario** (valutazioni psicometriche e score pesato) e **Amministratore** (dati di gestione privilegiata). L'architettura deve garantire che l'autenticazione (chi sei) non implichi automaticamente l'autorizzazione (cosa puoi vedere). Un accesso diretto tramite i metodi CRUD standard (es. findById) presenta un rischio di sicurezza significativo:

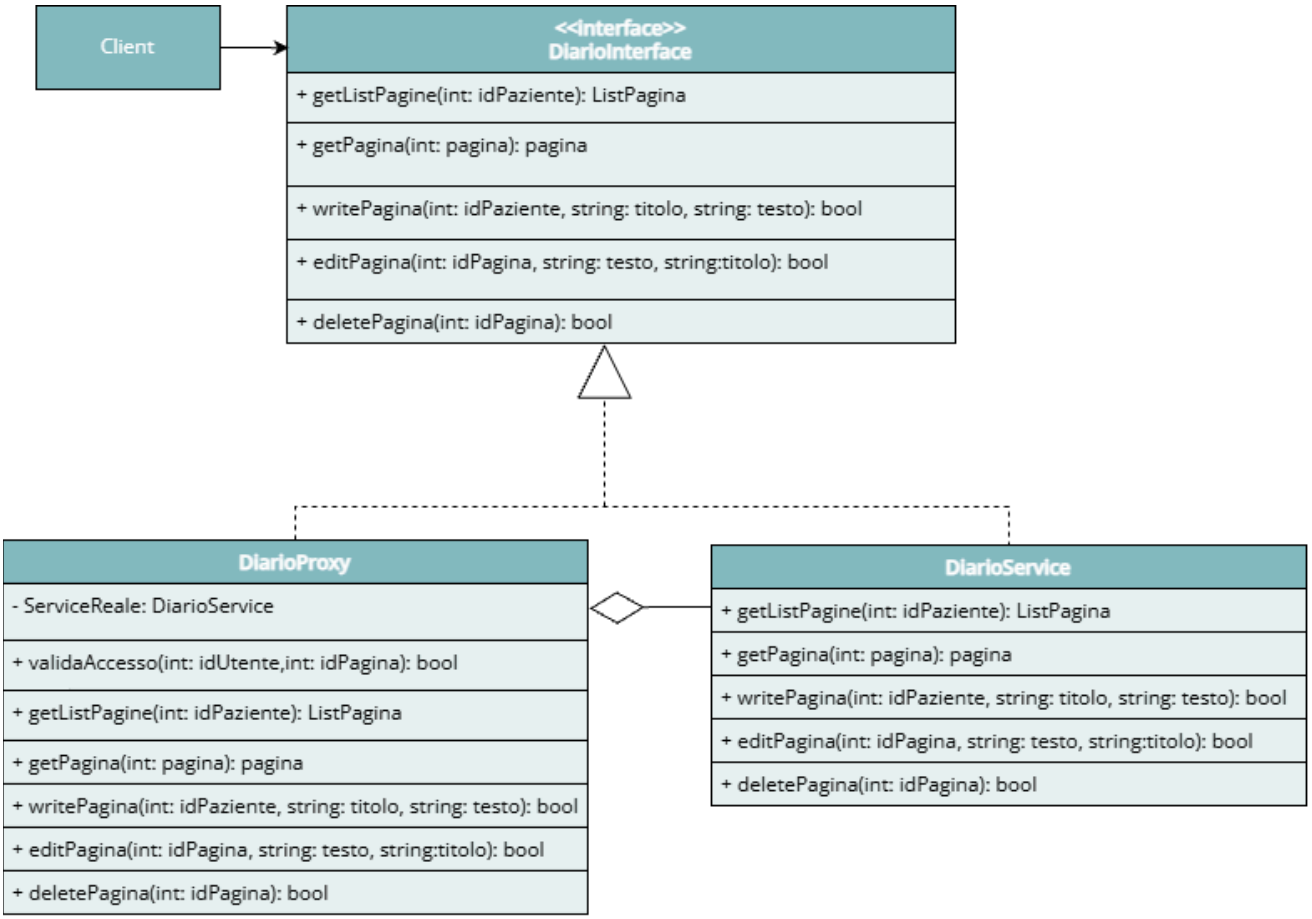
- **Violazione della Privacy**: Senza un controllo intermedio, un utente malintenzionato o un bug nel frontend potrebbe permettere a un paziente di richiedere tramite ID il diario o il questionario di un altro utente.
- **Granularità dei Permessi**: La logica di accesso è complessa (es. uno Psicologo può vedere solo i diari dei suoi pazienti assegnati, non di tutti). Implementare questi controlli ripetitivamente in ogni metodo del Service creerebbe codice ridondante e difficile da manutenere.

Per risolvere questa criticità di sicurezza, è stato applicato il design pattern strutturale Proxy (nella variante Protection Proxy). Questo pattern fornisce un surrogato o un segnaposto per un altro oggetto al fine di controllarne l'accesso, permettendo di eseguire operazioni preliminari prima che la richiesta raggiunga l'oggetto reale.

Nel sistema SINTONIA, il Proxy agisce come un "guardiano" tra il livello dei Controller e il livello di accesso ai dati (Service). Intercettando le chiamate verso le entità **Paziente, Psicologo, Pagina_Diario, Questionario** e **Amministratore**, il Proxy verifica dinamicamente le regole di business (es. Ownership Check: "L'utente richiedente è il proprietario di questo diario?" o Relationship Check: "Lo psicologo richiedente ha in cura questo paziente?"). Solo se i controlli di sicurezza sono superati, la richiesta viene inoltrata all'oggetto reale per il recupero dei dati, in caso contrario, viene sollevata un'eccezione di sicurezza, proteggendo l'integrità e la riservatezza del database.

DI seguito vi è una rappresentazione grafica del design pattern proxy applicata alla logica dell'entità **Pagina_Diario.** In egual modo lo stesso pattern verrà applicato alle altre entità sopracitate.

SINTONIA
La mente trova pace, un passo alla volta



Per una visualizzazione ottimale, consultare il seguente link.

SINTONIA
La mente trova pace, un passo alla volta

Glossario

Termine	Definizione
Client	È il componente (software o hardware) che invia una richiesta a un server per ottenere un servizio o un dato. Ad esempio, il browser che stai usando ora è un *client* web.
Python	È un linguaggio di programmazione ad alto livello e interpretato. È progettato per essere versatile, potente e facile da leggere grazie a una sintassi pulita.
Wrapper	È un codice che "avvolge" un altro pezzo di software (come una libreria o una funzione complessa) per semplificarne l'uso, tradurne i dati o renderlo compatibile con un altro sistema.
DAO (Data Access Object)	Pattern architetturale che fornisce un'interfaccia astratta verso il database, separando la logica di business dai dettagli specifici della persistenza dei dati.
DTO (Data Transfer Object)	Oggetto che trasporta dati tra processi o layer dell'applicazione. In SINTONIA è usato per gestire i dati tipizzati nella logica di business.
NLP (Natural Language Processing)	Elaborazione del Linguaggio Naturale. Campo dell'intelligenza artificiale utilizzato nel modulo Python per analizzare i testi (es. diario) e generare report.
JSON (JavaScript Object Notation)	Formato leggero per lo scambio di dati basato su testo. È il formato richiesto dal modulo Python per l'input strutturato.
Service	Componente che incapsula la logica di business (Business Logic) dell'applicazione. Si posiziona tra i Controller e il livello di accesso ai dati.
Controller	Componente del framework (NestJS) che gestisce le richieste in ingresso, invoca la logica di business e restituisce le risposte al client. Nel sistema, interagisce spesso con il Facade.